

Aprendizaje Automático

Guía de estudio para el examen

Resumen completo del temario (Temas 1–6 y 9) · Curso 2025–2026

Índice

1. **Tema 1.** Conceptos básicos del aprendizaje automático
2. **Tema 2.** Aprendizaje supervisado: evaluación y modelos (kNN, regresión lineal y logística, Naïve Bayes, SVM, árboles, ensembles, multclasificadores)
3. **Tema 3.** Calidad de datos y preprocesamiento
4. **Tema 4.** Aprendizaje no supervisado (clustering y detección de anomalías)
5. **Tema 5.** Aprendizaje profundo: fundamentos
6. **Tema 6.** Aprendizaje profundo en distintos datos: CNN y Transformers
7. **Tema 9.** Aspectos avanzados: XAI, AI Safety/OOD, aprendizaje continuo, unlearning, causalidad
8. **Apéndice.** Fórmulas clave y preguntas de repaso

Tema 1 · Conceptos básicos del aprendizaje automático

1.1 ¿Qué es el aprendizaje automático?

- Ciencia (y arte) de programar ordenadores para que **aprendan de los datos** sin ser programados explícitamente (*Arthur Samuel, 1959*).
- Definición ingenieril (*Tom Mitchell, 1997*): un programa aprende de la experiencia **E** respecto a una tarea **T** y una medida de rendimiento **P**, si su rendimiento en T (medido por P) mejora con E.
- Ejemplo filtro de spam: T = marcar spam; E = correos de entrenamiento etiquetados; P = precisión de clasificación.

¿Por qué usar ML?

- Problemas con muchas reglas/ajustes manuales → ML simplifica el código.
- Problemas complejos sin buena solución tradicional.
- Entornos cambiantes (el sistema se adapta a nuevos datos).
- Descubrir patrones en grandes volúmenes (**minería de datos**).

1.2 Formalización

Datos: $(x_i, y_i), i = 1..m$ | Modelo: $f(x, \omega), \omega \in \Omega$ | L = función de pérdida/error | R = riesgo (pérdida promedio)

Aprender = estimar $f(x, \omega_0)$ que **minimiza el riesgo** sobre el conjunto de funciones soportadas, usando solo los datos de entrenamiento (la distribución $p(x,y)$ es desconocida).

1.3 Tipos de aprendizaje

Tipo	Descripción	Tareas / algoritmos
Supervisado	Datos con etiquetas (soluciones deseadas)	Clasificación (spam, MNIST), Regresión (precio coche). kNN, regresión lineal/logística, SVM, árboles, random forest, redes neuronales
No supervisado	Datos sin etiquetar; aprende sin "profesor"	Clustering (k-Means, DBSCAN, HCA), detección de anomalías, reducción de dimensión (PCA, t-SNE), reglas de asociación (Apriori, Eclat)
Semi-supervisado	Pocos datos etiquetados + muchos sin etiquetar	Combinación de ambos
Por refuerzo	Un agente observa el entorno, ejecuta acciones y recibe recompensas ; aprende una política	AlphaGo, juegos Arcade

Basado en instancias vs. basado en modelos

- **Basado en instancias:** aprende los ejemplos "de memoria" y generaliza por una medida de similitud (distancia). No requiere entrenamiento (p. ej. 1-NN: $f(x)$ = etiqueta del ejemplo más cercano).
- **Basado en modelos:** construye un modelo de los ejemplos y lo usa para predecir.

1.4 Entorno general de ML y evaluación

- **Entrenamiento + test:** el modelo se entrena con el conjunto de entrenamiento y se evalúa con el de test. La tasa de error en casos nuevos = **error de generalización** (fuera de muestra). Reparto típico 80/20.
- **Validación cruzada k-fold (k-fcv):** se parte el conjunto en k bloques; se entrena con k-1 y se valida con el restante, rotando.
- **Criterios de evaluación:** precisión/exactitud, velocidad (construcción y uso), robustez (valores desconocidos), escalabilidad, **interpretabilidad**, complejidad del modelo.

1.5 Retos del ML en entornos reales

- **Teorema "No Free Lunch" (Wolpert):** sin suposiciones sobre los datos, ningún modelo es a priori mejor que otro. Hay que probar varios modelos razonables.
- **Sobreajuste (overfitting):** el modelo va bien en entrenamiento pero generaliza mal. Solución: **regularización** (restringir el modelo).
- **Subajuste (underfitting):** el modelo es demasiado simple.
- **Selección de modelos / ajuste de hiperparámetros:** usar un conjunto de validación; ajustar solo con el test produce sobreajuste al test.
- **Datos no representativos: sesgo de muestreo;** los conjuntos de validación/test deben representar la producción real.
- **Cantidad insuficiente de datos:** incluso problemas sencillos requieren miles de ejemplos (Banko & Brill 2001: con suficientes datos, algoritmos muy distintos rinden casi igual).

- **Calidad de datos:** limpiar datos es esencial (los científicos de datos dedican gran parte del tiempo a ello).

Data-Centric vs Model-Centric: Model-Centric mejora el modelo manteniendo los datos fijos; **Data-Centric** mejora sistemáticamente la **calidad de los datos** manteniendo el modelo. Estamos en la era del big data: el dato es "el petróleo del s. XXI".

1.6 IA fiable y regulación

- **AI Act** (marco regulatorio europeo) y enfoque basado en riesgo.
- **IA (con)fiable / Trustworthy AI:** lícita, ética y robusta.
- **AI Safety:** seguridad, robustez frente a adversarios (ejemplo: parches/"jerséis" adversarios que hacen invisible a una persona ante detectores).
- Ciclo de vida de la IA y gobernanza (Naciones Unidas, etc.).

Tema 2 · Aprendizaje supervisado: evaluación y modelos

2.1 Evaluación de modelos

Métricas de regresión

$RMSE(X, h) = \sqrt{(1/m) \sum (h(x^{(i)}) - y^{(i)})^2}$ → norma ℓ_2 , penaliza más los errores grandes
 $MAE = (1/m) \sum |h(x^{(i)}) - y^{(i)}|$ → norma ℓ_1 , mejor con outliers

Normas: ℓ_2 (euclídea), ℓ_1 (Manhattan), ℓ_0 (nº de no nulos), ℓ_∞ (máximo absoluto).

Métricas de clasificación (matriz de confusión)

$Precisión = TP / (TP + FP)$ Recall (sensibilidad, TPR) = $TP / (TP + FN)$ $F1 = 2 \cdot (Precisión \cdot Recall) / (Precisión + Recall)$ → media armónica
 $FPR = FP / (FP + TN)$
 $TNR = 1 - especificidad$

- **Compromiso precisión/recall:** el clasificador usa una **función de decisión** y un **umbral**. Subir el umbral ↑precisión y ↓recall; bajarlo ↓precisión y ↑recall.
- **Curva ROC:** representa TPR (recall) frente a FPR. **AUC** = área bajo la curva (1 = perfecto, 0.5 = azar).
- F1 favorece clasificadores con precisión y recall altos a la vez.

Clasificación multiclase

- **OvA / OvR (uno-contra-todos):** N clasificadores binarios (uno por clase); gana la mayor puntuación. Preferido en general.
- **OvO (uno-contra-uno):** $N(N-1)/2$ clasificadores (cada par); gana quien gane más duelos. Mejor para algoritmos que escalan mal con el tamaño (p. ej. SVM), pues cada clasificador usa menos datos.

2.2 Aprendizaje basado en instancias: k-NN

- **1-NN:** asigna la clase del ejemplo de entrenamiento más cercano según una función de distancia.
- **k-NN:** vota entre los k vecinos más cercanos (en regresión predice la media del vecindario).
- Distancias: euclídea (ℓ_2), Manhattan (ℓ_1), **Mahalanobis** (con matriz de covarianza Σ), L_∞ . La **escala/ponderación** de las características afecta a las regiones de decisión → conviene normalizar.
- Sin fase de entrenamiento; coste en predicción. Problemas en las fronteras.

2.3 Regresión lineal

$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T x$ (con $x_0 = 1$)

Entrenamiento = minimizar el **MSE**. Dos vías:

- **Ecuación Normal:** $\theta = (X^T X)^{-1} X^T y$ (solución cerrada). Implica invertir matriz ($O(n^2 \cdot 4 - n^3)$); puede no existir si $X^T X$ es singular. Se usa la **pseudoinversa** (vía SVD: $X = U \Sigma V^T$, $X^+ = V \Sigma^+ U^T$), siempre definida y más eficiente ($\approx O(n^2)$). Lineales en m.
- **Descenso de gradiente:** escala bien con muchas características.

Descenso de gradiente (GD)

$$\nabla_{\theta} \text{MSE}(\theta) = (2/m) X^T(X\theta - y) \quad \theta \leftarrow \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

- **η = tasa de aprendizaje:** muy pequeña $\rightarrow$ lento; muy grande $\rightarrow$ diverge. Importante **escalar las características** (StandardScaler).
- El MSE de la regresión lineal es **convexo** $\rightarrow$ un único mínimo global (sin mínimos locales).
- **Batch GD:** usa todo el conjunto en cada paso (lento con muchos datos, pero escala con n° de características).
- **Estocástico (SGD):** una instancia aleatoria por paso; rápido y permite datos enormes, pero "rebota". La aleatoriedad ayuda a escapar de óptimos locales; se reduce η con un **learning schedule**.
- **Mini-batch GD:** grupos pequeños; aprovecha optimización hardware (GPU), menos errático que SGD.

2.4 Regresión polinómica, curvas de aprendizaje y sesgo-varianza

- **Regresión polinómica:** añadir potencias de las características y entrenar un modelo lineal $\rightarrow$ ajusta datos no lineales. Grados altos $\rightarrow$ sobreajuste.
- **Curvas de aprendizaje:** error de entrenamiento y de validación frente al tamaño/iteraciones. Sobreajuste = buen entrenamiento, mala validación; subajuste = mal en ambos.

Descomposición del error de generalización = Sesgo ² + Varianza + Error irreducible				
• Sesgo:	suposiciones	erróneas	(modelo demasiado simple)	$\rightarrow$ underfitting.
• Varianza:	sensibilidad excesiva a variaciones del entrenamiento	(modelo complejo)	$\rightarrow$ overfitting.	
• Error irreducible:	ruido de los datos	(solo se reduce limpiando).		
$\uparrow$ complejidad $\Rightarrow \uparrow$ varianza y $\downarrow$ sesgo (y viceversa): por eso es un <i>equilibrio</i> .				

2.5 Modelos lineales regularizados

Modelo	Término de regularización	Característica
Ridge	$\alpha \sum \theta_i^2$ (norma ℓ_2)	Reduce pesos; no los anula. Buena opción por defecto
Lasso	$\alpha \sum \theta_i $ (norma ℓ_1)	Selección automática de características (pone pesos a 0 $\rightarrow$ modelo disperso)
Elastic-Net	mezcla ridge + lasso (ratio r)	Intermedio; preferible a Lasso si hay correlación o $n > m$

- El término de regularización solo se añade **durante el entrenamiento**. El sesgo θ_0 no se regulariza.
- **Early stopping:** detener cuando el error de validación alcanza el mínimo (regularización de algoritmos iterativos).
- Casi siempre conviene algo de regularización; evitar la regresión lineal pura.

2.6 Regresión logística y softmax

$$p = h_{\theta}(x) = \sigma(\theta^T x), \quad \sigma(t) = 1/(1+e^{-t}) \quad (\text{sigmoide}). \quad \text{Predice 1 si } p \geq 0.5 \quad (\theta^T x \geq 0)$$

- **Coste (log loss):** $J(\theta) = -(1/m) \sum [y \log(\hat{p}) + (1-y) \log(1-\hat{p})]$. Convexa $\rightarrow$ GD encuentra el mínimo global

(no hay ecuación normal).

- **Regresión Softmax (logística multinomial):** generaliza a K clases. Calcula una puntuación $s_k(x) = \theta^{(k)T}x$ por clase y aplica softmax: $p_k = e^{s_k} / \sum e^{s_j}$. Predice la clase de mayor probabilidad. Coste = **entropía cruzada**.

2.7 Naïve Bayes

$$c_{\text{MAP}} = \operatorname{argmax}_c P(c \mid a_1, \dots, a_n) = \operatorname{argmax}_c P(c) \cdot \prod_i P(a_i \mid c)$$

- Modelo bayesiano más sencillo: supone **independencia condicional** de todos los atributos dada la clase (suposición "ingenua").
- Pese a ser irreal, es muy competitivo.
- **Variables discretas:** $P(x|c)$ por frecuencia relativa (MLE). **Suavizado de Laplace:** $p = (n(x,c)+1)/(n(c)+|X|)$ para evitar ceros.
- **Variables numéricas:** se asume distribución gaussiana $N(\mu, \sigma)$ por clase.

2.8 Máquinas de Vector Soporte (SVM)

- Busca el **hiperplano de margen máximo** que separa las clases → buena generalización. Las instancias en el borde del margen son los **vectores de soporte**.
- **Margen duro:** todas las instancias fuera de la "calle". Solo si son linealmente separables; sensible a outliers.
- **Margen blando:** equilibrio entre margen amplio y violaciones (variables de holgura ζ). Hiperparámetro **C**: $\downarrow C \rightarrow$ más regularización (más violaciones permitidas).
- Sensible a la **escala** de las características.

Truco del kernel

- Permite obtener el efecto de añadir muchas características (incluso de dimensión infinita) **sin calcularlas**, usando el problema dual (solo aparecen productos escalares).
- Kernels: **lineal**; **polinómico** $K(a,b)=(a^Tb+c)^d$; **RBF gaussiano** $K(a,b)=\exp(-\gamma\|a-b\|^2)$; sigmoide.
- **Teorema de Mercer:** si K cumple las condiciones de Mercer (continua, simétrica), existe ϕ tal que $K(a,b)=\phi(a)^T\phi(b)$, aunque no se conozca ϕ .
- **Guía práctica:** probar primero kernel lineal (LinearSVC, rápido $O(m \cdot n)$); si no, RBF. SVC (con kernel) es $O(m^2 \cdot n)$ – $O(m^3 \cdot n)$; malo para conjuntos grandes.
- **SVR (regresión SVM):** ajusta el máximo de instancias **dentro** de la calle de anchura ϵ (ϵ -insensible).

2.9 Árboles de decisión

- Clasificación y regresión. Modelo de **caja blanca** (interpretable). Genera reglas Si-Entonces.
- Construcción **divide y vencerás** (voraz): se elige en cada nodo el atributo que **mejor separa**; recursión hasta que el nodo es puro o no quedan atributos.

Criterios de selección de atributo

$$\text{Entropía: } H(S) = - \sum p_i \log_2 p_i \quad \text{Ganancia de información (ID3): } \text{Gain}(S,A) = H(S)$$

$-\sum (|S_v|/|S|) H(S_v)$ Gain Ratio (C4.5): $\text{Gain}(S,A) / H(A)$ (penaliza atributos con muchos valores) Índice Gini (CART): $\text{Gini} = 1 - \sum p_j^2$

- **ID3**: máxima ganancia de información; tiende a elegir atributos con muchos valores; sin poda; óptimo local (hill-climbing).
- **C4.5**: mejora ID3 → gain ratio, datos continuos por rangos, valores perdidos, **poda** (pre-poda y post-poda: reemplazo y elevación de subárbol).
- **CART (scikit-learn)**: solo árboles **binarios**; minimiza impureza ponderada (Gini por defecto) en clasificación, MSE en regresión. Encontrar el árbol óptimo es NP-completo.
- **Regularización**: max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, max_features. Aumentar min_* o reducir max_* regulariza. Son modelos **no paramétricos**.
- **Limitaciones**: límites ortogonales (sensibles a rotación → usar PCA), **alta varianza** (pequeños cambios → árboles muy distintos).

2.10 Ensembles y Random Forests

"Sabiduría de la multitud": agregar muchos predictores suele superar al mejor individual. Un conjunto de **aprendices débiles** puede formar un **aprendiz fuerte** si son diversos.

Método	Idea
Voting	Combina distintos algoritmos. <i>Hard voting</i> : mayoría de votos. <i>Soft voting</i> : media de probabilidades
Bagging	Mismo algoritmo, subconjuntos aleatorios con reemplazo (bootstrap). Reduce varianza. Paralelizable. OOB : ~37% de instancias no muestreadas sirven para validar
Pasting	Igual pero sin reemplazo
Random Forest	Ensemble de árboles con bagging + subconjunto aleatorio de características por nodo. Mide importancia de características
Extra-Trees	Además, umbrales aleatorios por característica → más rápido, más sesgo / menos varianza
Boosting	Predictores secuenciales , cada uno corrige al anterior. No paralelizable
Stacking	Un meta-modelo (blender) aprende a combinar las predicciones de los predictores base
	• AdaBoost: aumenta el peso de las instancias mal clasificadas; cada predictor se centra en los casos difíciles; los predictores se ponderan por su precisión. Versión multiclase: SAMME / SAMME.R. • Gradient Boosting (GBRT): cada predictor ajusta los errores residuales del anterior. HistGradientBoosting (HGB): discretiza en bins → muy rápido en grandes datos. Implementaciones: XGBoost, LightGBM, CatBoost.

Descomposición multclasificadores (agregación)

- **OvO**: $c(c-1)/2$ binarios. Agregación: *voto mayoritario (MV)* o *voto ponderado (WV)*. Cuidado con clasificadores no competentes.
- **OvA/OvR**: c binarios (una clase vs resto). Agregación *confianza máxima (MAX)*. Desventaja: fuerte desequilibrio de clases.

- **Clasificación no balanceada:** los clasificadores estándar sobreaprenden la clase mayoritaria; hay que cambiar la evaluación (no usar solo accuracy) y/o reequilibrar.

Tema 3 · Calidad de datos y preprocesamiento

3.1 Importancia del preprocesamiento

- Transformar datos brutos en "**smart data**" (datos de calidad). Decisiones de calidad $\Leftarrow$ datos de calidad.
- Puede consumir ~80% del tiempo de un proyecto de minería de datos.
- Datos reales son impuros: **incompletos** (valores perdidos), con **ruido**, **inconsistentes**.
- Engloba: **preparación** (integración, limpieza, normalización, transformación) y **reducción** (selección de características/instancias, discretización).

3.2 Integración, limpieza y transformación

- **Integración:** combinar fuentes; detectar duplicados, inconsistencias y **redundancia** (atributo derivable de otros; análisis de correlaciones).
- **Limpieza:** resolver inconsistencias, suavizar ruido, eliminar outliers.
- **Transformación:** agregación, generalización (jerarquías de conceptos), normalización.

Normalización

Min-max: $v' = (v - \min) / (\max - \min) \cdot (\text{nuevoMax} - \text{nuevoMin}) + \text{nuevoMin}$ Z-score
(zero-mean): $v' = (v - \mu) / \sigma$ Escala decimal: $v' = v / 10^j$

Útil para métodos basados en distancias o redes neuronales. Z-score útil si hay outliers o límites desconocidos.

3.3 Datos imperfectos

Valores perdidos

- Ignorar la tupla; rellenar manualmente; constante global ("?"); media/desviación (global o por clase); valor más probable (inferencia bayesiana o árbol); relaciones entre variables (regresión).
- Imputación por ML: **KNNI** (k-NN), **WKNNI** (ponderado), **KMI** (clustering), **SVMI**, EC, SVDI, LLSI.

Ruido

- Tipos de ejemplos: **safe**, **borderline**, **noisy**. Interacciones: small disjuncts, solapamiento de clases.
- **Filtros de ruido** (esquema de votación para eliminar ejemplos mal etiquetados):
 - **Ensemble Filter (EF):** varios algoritmos (C4.5, 1-NN, LDA) etiquetan vía k-fcv; voto por *consenso* (todos fallan) o *mayoría*.
 - **Cross-Validated Committees Filter (CVCF):** mismo algoritmo (C4.5); cada clasificador etiqueta TODOS los ejemplos.
 - **Iterative Partitioning Filter (IPF):** aplica CVCF en varias iteraciones hasta criterio de parada.

Outliers

Valores anómalos pero correctos. Detección: distancia media alta al resto; clustering parcial (lo que queda fuera).

3.4 Discretización

- Transforma valores numéricos ordenados en intervalos nominales. Es también una técnica de reducción.
- **No supervisados:** igual amplitud, igual frecuencia, clustering (k-medias). Igual amplitud puede dar desequilibrios.
- **Supervisados (usan la clase):** basados en entropía (**Fayyad & Irani, MDLP**), métodos Chi-square (**ChiMerge**).
- Ejes de clasificación: supervisado/no supervisado, dinámico/estático, local/global, top-down/bottom-up, directo/incremental.
- Evaluación: nº de intervalos, nº de inconsistencias, tasa de acierto.

3.5 Selección de características (FSS)

Encontrar el subconjunto de variables que optimiza la clasificación. Menos datos → aprende más rápido; mayor exactitud; resultados más simples. Espacio de búsqueda: 2^N (búsqueda exhaustiva inviable).

Funciones objetivo

- **Filtro (filter):** evalúa por la información de los datos (separabilidad, correlación, **información mutua** $I(X;C)=H(C)-H(C|X)$, consistencia). Rápido pero tiende a incluir muchas variables.
- **Envolvente (wrapper):** evalúa entrenando el propio clasificador (% acierto). Más preciso pero **muy costoso** y sesgado al clasificador usado.

Estrategias de búsqueda

- **Forward (hacia delante):** empieza vacío y añade la mejor variable. Bueno si el subconjunto óptimo es pequeño; no elimina.
- **Backward (hacia atrás):** empieza completo y elimina la menos útil. Bueno si el óptimo es grande.
- **l-más r-menos:** generaliza ambos. Branch & bound, beam search (exponenciales).
- **Estocásticos:** hill-climbing con reinicios, enfriamiento simulado, algoritmos genéticos.
- Algoritmos: Focus, MIFS, LVF, LVW, **Relief**, **mRMR** (mínima redundancia, máxima relevancia).
- Salida: **ranking** de atributos o **subconjunto** de atributos.

3.6 Selección de instancias (SI)

- Elegir ejemplos relevantes → menos datos, mejor generalización, resultados más simples.
- Tipos: **muestreo**, **selección de prototipos** (para kNN), aprendizaje activo.
- Propiedades: dirección (incremental, decremental, batch, mixta), tipo (**condensación**, **edición**, híbrido), evaluación (filtro/wrapper).
- **CNN (Condensed NN):** condensación incremental; retiene puntos del borde.
- **ENN (Edited NN):** edición; elimina instancias mal clasificadas por sus k vecinos; suaviza fronteras.
- Híbridos: **DROP3**; evolutivos: **SSMA**, **CHC**. Coste alto ($O(n^2)-O(n^3)$) → para grandes BD se usa **estratificación**.

3.7 Reducción de dimensionalidad

- **Maldición de la dimensionalidad:** en alta dimensión los puntos están muy dispersos y lejanos → predicciones poco fiables, riesgo de sobreajuste. Los datos necesarios crecen exponencialmente con las dimensiones.
- **Hipótesis del manifold:** los datos reales de alta dimensión se aproximan a una variedad (manifold) de mucha menor dimensión.

Técnicas

- **Proyección:** proyectar a un subespacio de menor dimensión. No sirve si el manifold está retorcido (rollo suizo).
- **PCA (Análisis de Componentes Principales):** encuentra los ejes (componentes principales) que **maximizan la varianza**, ortogonales entre sí, vía **SVD** ($X = U\Sigma V^T$). Proyecta a las d primeras componentes. Ej.: MNIST con 95% de varianza → 154 de 784 características. **Error de reconstrucción** al descomprimir. Variantes: PCA incremental (IPCA), PCA aleatorio.
- **Proyección aleatoria:** proyección lineal aleatoria que conserva las distancias (Johnson-Lindenstrauss); rápida, eficiente, buena en alta dimensión.
- **LLE (Local Linear Embedding):** no lineal, basado en manifold; conserva relaciones lineales locales entre vecinos. Bueno para desenrollar manifolds; escala mal ($O(m^2)$).
- **Otras:** MDS (conserva distancias), Isomap (distancias geodésicas), **t-SNE** (visualización, mantiene cercanos juntos), **LDA** (supervisado, ejes más discriminantes entre clases).

Tema 4 · Aprendizaje no supervisado

4.1 Técnicas

- **Clustering:** agrupar instancias similares.
- **Detección de anomalías/novedades:** aprender lo "normal" (inliers) y detectar lo anómalo.
- **Estimación de densidad:** estimar la PDF del proceso generador (útil para anomalías).

4.2 k-Means

- Algoritmo: inicializar k centroides; repetir hasta convergencia → (1) asignar cada instancia al centroide más cercano, (2) recalcular centroides como la media de sus instancias.
- Converge, pero puede caer en un **óptimo local** (depende de la inicialización).
- **k-means++:** inicialización inteligente que elige centroides distantes (probabilidad $\propto D(x)^2$) → reduce óptimos subóptimos.
- **Mini-batch k-means:** usa minilotes; más rápido, inercia algo peor.
- **Inercia:** suma de distancias² instancia-centroide. No sirve directamente para elegir k (siempre baja al subir k).

Elegir el número de clusters

- **Método del codo:** graficar inercia vs k y buscar el "codo".
- **Coefficiente de silueta:** $(b - a) / \max(a, b)$, con a = distancia media intra-cluster, b = distancia media al cluster más cercano. Rango [-1, 1]; cerca de +1 = bien asignado.

Límites de k-Means: hay que fijar k y ejecutar varias veces; va mal con clusters de distinto tamaño, densidad o forma no esférica.

Otras medidas de calidad de clustering

- **Davies-Bouldin:** dispersión interna / separación entre centroides. Menor = mejor.
- **Dunn:** distancia mínima entre clusters / diámetro máximo. Mayor = mejor.
- **Pseudo-F:** distancia entre-cluster / intra-cluster. Mayor = mejor.

4.3 DBSCAN y otros

- **DBSCAN:** clusters = regiones densas. Para cada instancia cuenta vecinos dentro de radio ϵ ; si tiene $\geq \text{min_samples}$ es **instancia central**. Las instancias en la vecindad de una central forman cluster; el resto, anomalías. Robusto a outliers, encuentra cualquier forma, solo 2 hiperparámetros; falla si las densidades varían mucho; $O(m^2n)$. **HDBSCAN** mejora densidades variables.
- **Agglomerativo (jerárquico):** fusiona los clusters más cercanos de abajo a arriba → árbol/dendrograma.
- **BIRCH:** para datos muy grandes (pocas características), construye un árbol resumen.
- **Mean-shift:** desplaza círculos hacia la mayor densidad; encuentra cualquier n°/forma; $O(m^2n)$.
- **Affinity Propagation:** instancias eligen "exemplars"; no fija k de antemano; $O(m^2)$.

- **Clustering espectral:** embedding de baja dimensión de la matriz de similitud + clustering; capta estructuras complejas.

4.4 Detección de anomalías

Identificar patrones raros (fraude, fallos, intrusiones, enfermedades raras). Retos: definir la región normal, frontera difusa, datos etiquetados escasos, ruido, comportamiento normal cambiante, adversarios.

Tipos de anomalías

- **Puntuales:** una instancia individual anómala.
- **Contextuales:** anómala solo en un contexto.
- **Colectivas:** una colección de instancias relacionadas (las individuales no son anómalas por sí solas).

Técnicas

- **Basadas en distancia (k-NN):** outliers = puntos con mayor distancia al k-ésimo vecino. Mal con densidades variables.
- **Basadas en densidad (LOF, Local Outlier Factor):** compara la densidad local de una instancia con la de sus vecinos; densidad mucho menor = outlier. Detecta lo que el enfoque NN no capta.
- **Basadas en modelo (One-Class SVM):** hiperplano que separa la clase objetivo del origen con margen máximo (o hiperesfera mínima). $f(x) < 0 \Rightarrow$ anomalía.
- **Basadas en ensembles (Isolation Forest):** árboles de aislamiento (iTrees) con cortes aleatorios; las anomalías se **aíslan antes** (menor profundidad media) $\rightarrow$ mayor puntuación de anomalía.

Tema 5 · Aprendizaje profundo: fundamentos

5.1 De la neurona biológica al perceptrón

- **McCulloch & Pitts (1943)**: primera neurona artificial (entradas/salida binarias).
- **Perceptrón (Rosenblatt, 1957)**: neurona TLU/LTU. Calcula $z = w^T x + b$ y aplica función escalón (Heaviside). Clasificador lineal binario.
- Una capa de TLUs = capa densa/totalmente conectada. Regla de aprendizaje (Hebb): $w_{i,j} \leftarrow w_{i,j} + \eta(\hat{y}_j - y_j)x_i$... refuerza conexiones que reducen el error.
- **Limitación**: no resuelve problemas no lineales (XOR) (Minsky & Papert, 1969). Solución: **perceptrón multicapa (MLP)**.

5.2 MLP y retropropagación

- **MLP**: capa de entrada + una o más **capas ocultas** + capa de salida.
- **Retropropagación (backprop)**: auto-diff en modo inverso + descenso de gradiente. Por cada mini-batch:
 1. **Forward pass**: calcula salidas capa a capa (guardando intermedios).
 2. Mide el error (función de pérdida).
 3. **Backward pass**: propaga el gradiente del error hacia atrás (regla de la cadena), midiendo la contribución de cada peso/sesgo.
 4. Paso de descenso de gradiente para ajustar pesos.
- Importante **inicializar los pesos aleatoriamente**. Se sustituyó la función escalón por funciones con derivada no nula (sigmoide).

Funciones de activación

- **Sigmoide**: $\sigma(z) = 1/(1+e^{-z})$, salida (0,1).
- **tanh**: salida (-1,1), centrada en 0 (mejor convergencia).
- **ReLU**: $\max(0, z)$; rápida; por defecto. No diferenciable en 0.

MLP para regresión y clasificación

- **Regresión**: 1 neurona de salida por valor; sin activación (o adecuada). Pérdida: MSE.
- **Clasificación binaria**: 1 salida con sigmoide. **Multiclase exclusiva**: 1 salida por clase + **softmax**. Pérdida: entropía cruzada.

Ecosistema: **Keras** (API de alto nivel de TensorFlow), PyTorch, JAX. Modelos secuenciales y API funcional (p. ej. **Wide & Deep**: conecta entradas directamente a la salida; aprende patrones profundos y reglas simples).

5.3 Ajuste de hiperparámetros

- **Nº de capas**: empezar con 1; las redes profundas son más eficientes en parámetros (aprenden jerarquías) → mejor generalización y **transfer learning**.
- **Nº de neuronas**: mismo nº por capa suele funcionar; enfoque "pantalones elásticos" (red grande + early

stopping).

- **Tasa de aprendizaje:** el hiperparámetro **más importante**. Buscarla aumentando η exponencialmente y observando la pérdida.
- **Tamaño de lote:** grande aprovecha GPU pero puede dar inestabilidad; **activación:** ReLU por defecto en ocultas; **iteraciones:** usar early stopping.
- Herramientas: GridSearch/RandomizedSearch, **Keras Tuner**, AutoML, algoritmos evolutivos.

5.4 Entrenamiento de redes profundas (DNN)

Gradientes inestables

- **Vanishing gradients (gradientes que desaparecen):** al retropropagar, los gradientes se hacen diminutos en capas inferiores → no aprenden. Causa: sigmoide + mala inicialización (saturación).
- **Exploding gradients:** gradientes crecen sin control (frecuente en RNN).

Soluciones

- **Inicialización:** **Glorot** (sigmoide/tanh), **He/Kaiming** (ReLU y variantes), **LeCun** (SELU). Mantienen la varianza de señal y gradiente.
- **Mejores activaciones:** **Leaky ReLU** ($\max(\alpha z, z)$, evita ReLU moribundas), **PReLU** (α aprendible), **ELU/SELU** (SELU autonormaliza MLPs), **GELU**, **Swish** ($z\sigma(\beta z)$, mejores en tareas complejas), **Mish**.
Guía: ReLU para tareas simples (rápida); **Swish/GELU** para tareas complejas; SELU para MLPs profundos.
- **Batch Normalization (BN):** normaliza (centra y escala) las entradas de cada capa por mini-batch, con parámetros aprendibles γ (escala) y β (desplazamiento). Reduce gradientes inestables, acelera el entrenamiento, actúa como regularizador. En test usa medias/desviaciones móviles. Añade coste (fusionable tras entrenar).
- **Gradient Clipping:** recorta gradientes a un umbral (sobre todo en RNN).

5.5 Reutilización y preentrenamiento

- **Transfer learning:** reutilizar capas (inferiores) de una red preentrenada en tarea similar. Acelera y necesita menos datos. Congelar capas reutilizadas y descongelar gradualmente.
- **Preentrenamiento no supervisado:** con muchos datos sin etiquetar, entrenar un autoencoder/GAN y reutilizar sus capas. (Antes: greedy layer-wise con RBM, Hinton 2006).
- **Preentrenamiento en tarea auxiliar / autosupervisado:** generar etiquetas automáticamente (p. ej. enmascarar palabras y predecirlas).

5.6 Optimizadores más rápidos

Optimizador	Idea
Momentum	Acumula gradientes pasados (vector momento, $\beta \approx 0.9$); el gradiente actúa como aceleración. Escapa mesetas, hasta 10× más rápido

Nesterov (NAG)	Mide el gradiente "adelantado" en $\theta + \beta m$. Algo más rápido
AdaGrad	Tasa adaptativa: reduce η más en dimensiones empinadas. Se detiene demasiado pronto en DNN
RMSProp	Como AdaGrad pero con decaimiento exponencial ($\rho \approx 0.9$). Soluciona el problema de AdaGrad
Adam	Momentum + RMSProp (1er y 2º momento). $\eta \approx 0.001$ por defecto. Muy usado
AdaMax, Nadam, AdamW	Variantes de Adam (Nadam = Adam + Nesterov; AdamW = decaimiento de pesos correcto)

Optimización de 2º orden (Hessianos) inviable en DNN (n^2 por salida). Modelos dispersos: regularización ℓ_1 o poda (TF-MOT).

Programación de la tasa de aprendizaje (learning schedules)

- **Power scheduling:** $\eta(t) = \eta_0 / (1 + t/s)^c$.
- **Exponencial:** $\eta(t) = \eta_0 \cdot 0.1^{t/s}$.
- **Constante a trozos, Performance scheduling** (reduce al estancarse la validación), **1-cycle** (sube y baja η ; "superconvergencia").

5.7 Regularización contra el sobreajuste

- **Early stopping, Batch Norm** (también regulariza), regularización ℓ_1/ℓ_2 .
- **Dropout:** en cada paso, cada neurona (salvo las de salida) tiene probabilidad **p** de "apagarse". Fuerza redundancia y robustez → red más generalizable ($\approx$ ensemble de 2^N redes). Dividir pesos por $(1-p)$. Tasa típica 20–30% (RNN), 40–50% (CNN).
- **MC Dropout:** aplicar dropout también en test y promediar varias pasadas → mejor estimación de incertidumbre (aproximación bayesiana) sin reentrenar.
- **Max-Norm:** restringe $\|w\|_2 \leq r$ tras cada paso.

Tema 6 · Aprendizaje profundo en distintos datos: CNN y Transformers

6.1 Redes profundas y CNN

- **Red profunda:** red con muchas capas ocultas → aproxima funciones complejas y aprende **representaciones jerárquicas** (low-level → mid → high-level features).
- **CNN (ConvNet):** red con al menos una capa de **convolución**; para datos en rejilla (imágenes).

Convolución

- Operación local lineal con una **máscara/filtro/kernel**: se multiplica elemento a elemento, se suman los productos y se sustituye la posición central → **mapa de activación / feature map**.
- **Idea clave:** en las CNN los coeficientes del filtro **se aprenden** (son pesos), no los diseña un humano (LeCun 1989; LeNet-5, 1998).

Ventajas de las CNN frente a capas densas

- **Conectividad dispersa** (operación local) → muchas menos operaciones.
- **Compartición de parámetros (weight sharing):** el mismo filtro recorre toda la imagen → muchísimos menos parámetros (regularización).
- **Equivarianza a la traslación** (no al escalado ni rotación).
- Permiten (teóricamente) entradas de tamaño variable.

Elementos de una CNN

- Capas convolucionales intercaladas con **activaciones no lineales** (ReLU). La no linealidad es imprescindible (si todo fuera lineal, equivaldría a una sola capa lineal).
- Los filtros tienen la **misma profundidad** que el volumen de entrada.
- **Stride** (paso) y **padding** (relleno, p. ej. zero-padding para conservar tamaño).
| $\text{Tamaño salida} = ((N + 2P - F)/\text{stride}) + 1$ (N=entrada, F=filtro, P=padding)
| $\text{Parámetros por capa} = (F \cdot F \cdot \text{profundidad} + 1) \cdot \text{nº de filtros}$
- **Pooling** (max/average): reduce dimensionalidad e introduce invarianza a pequeñas traslaciones.
- **Flatten** → vector; **capas densas** finales; **Softmax** para probabilidades de clase.

ML clásico vs Deep Learning: el ML clásico separa *feature engineering* (diseño manual) + clasificación; las CNN hacen **feature learning** de extremo a extremo (end-to-end). La clave es una buena representación interna.

Autoencoders

Arquitecturas **encoder-decoder** que aprenden una codificación eficiente de la entrada. Usos: compresión, **denoising** (se corrompe la entrada con ruido y el modelo aprende a reconstruir la original).

6.2 Transformers (modelos basados en atención)

Por qué las secuencias son difíciles

- Secuencia = elementos + orden + contexto (texto, series temporales, audio, vídeo). Cada elemento se convierte en un **vector (embedding)**.
- **Redes densas**: tratan la entrada como bloque fijo, no aprovechan el orden, no generalizan a longitud variable.
- **CNN**: capturan localidad pero no siempre dependencias largas.
- **RNN**: procesan paso a paso con memoria (estado oculto); pierden información lejana en secuencias largas.
- **LSTM**: añaden puertas (olvido, entrada, salida) para regular la memoria, pero siguen siendo **secuenciales** (no paralelizables, lentas, cuello de botella en el estado oculto).

Mecanismo de atención

- Permite **conexiones directas** entre elementos: cada token consulta al resto. La salida es una **combinación ponderada** de toda la secuencia.
- De cada embedding se generan 3 vectores: **Query (Q)** = qué busco, **Key (K)** = qué ofrezco, **Value (V)** = qué información transmito.
- **Scaled Dot-Product Attention**: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) V$. $Q \cdot K$ calcula relevancia, se escala, softmax $\rightarrow$ pesos que suman 1, y se combinan los V.

Tipos y bloque Transformer

- **Self-attention**: Q, K, V de la misma secuencia $\rightarrow$ cada token incorpora información del resto (dependencias largas).
- **Cross-attention**: Q de una secuencia, K/V de otra (conecta generación con la entrada).
- **Máscaras**: en generación, un token solo ve los anteriores y el actual (no los futuros).
- **Multi-head attention**: varias atenciones en paralelo $\rightarrow$ distintos tipos de relación (sintáctica, semántica, posicional...).
- **Bloque Transformer**: Multi-head Attention + **Feed-Forward Network** (procesa cada token por separado), con **conexiones residuales** (no perder info, estabilizar gradiente) y **Layer Normalization**.
- **Positional encoding**: la atención no conoce el orden $\rightarrow$ se suma información posicional al embedding. Entrada = contenido + posición.
- **Encoder** (self-attention $\rightarrow$ representación contextual) y **Decoder** (masked self-attention + cross-attention $\rightarrow$ genera salida paso a paso).

Arquitecturas y propiedades

- **Encoder-only** (comprender/embeddings), **Decoder-only** (generación, LLMs), **Encoder-Decoder** (traducción/transformación).
- **Coste cuadrático** $O(T^2)$ en la longitud de la secuencia.
- **Ventajas**: dependencias a largo plazo, **procesamiento en paralelo** (clave para escalar a billones de parámetros), relaciones flexibles.
- **Limitaciones**: coste cuadrático, mucha memoria/datos, necesitan info posicional, correlaciones espurias,

difíciles de interpretar.

- **Aplicaciones:** LLMs (ChatGPT, Gemini, Claude, Llama, Mistral...), series temporales (Informer, Autoformer, PatchTST, Chronos), visión (**ViT**: imagen → parches → embeddings → Transformer).

Tema 9 · Aspectos avanzados, retos y desafíos

9.1 Inteligencia Artificial Explicable (XAI)

- **Definición:** técnicas que hacen comprensibles para humanos las decisiones de modelos "caja negra". Requisito de ingeniería para sistemas críticos.
- **Caso LIME (husky vs lobo):** 94% accuracy pero el modelo miraba la **nieve** (fondo), no el animal → *shortcut learning*. La precisión no basta: hay que auditar qué aprende.
- **Tres motivos:** **Confianza** (medicina, conducción), **Depuración** (sesgos, data leakage, shortcuts), **Cumplimiento** (EU AI Act, GDPR Art. 22 "right to explanation").

Taxonomía

- **RED XAI** (orientada al modelo: research, explore, debug) vs **BLUE XAI** (orientada al humano: confianza, ética, legal).
- **Pre-hoc** (modelo interpretable desde el diseño: árboles, reglas, lineales, ProtoPNet) vs **Post-hoc** (explicar la caja negra: LIME, SHAP, Integrated Gradients, LRP, CRP).
- **Local** (una predicción) vs **Global** (todo el modelo).
- **Agnóstico** (cualquier modelo: LIME, SHAP) vs **Específico** (acceso interno: Integrated Gradients, LRP, Grad-CAM).

Métodos

- **LIME:** perturba la entrada localmente y entrena un modelo simple que aproxime al clasificador en esa vecindad. Agnóstico pero **inestable**.
- **SHAP:** valores de Shapley (teoría de juegos); contribución de cada feature promediada sobre todos los subconjuntos. Única solución que cumple eficiencia, simetría, linealidad y dummy. Coste exponencial.
- **Contraejemplos (counterfactuals):** "si tu ingreso fuera 5.000€ mayor, sería aprobado". Intuitivo, alineado con razonamiento causal.
- **Integrated Gradients:** integra el gradiente desde una baseline hasta la entrada (redes neuronales).
- **LRP (Layer-wise Relevance Propagation):** propaga la relevancia hacia atrás capa a capa (heatmaps por píxel). **CRP** extiende a **conceptos** de alto nivel.
- **ProtoPNet:** "esto se parece a este prototipo" (explicación intrínseca, RED XAI).
- **Evaluación:** **Quantus** (+30 métricas: fidelidad, localización, complejidad, robustez); propuestas DAsCI: REVEL, X-SHIELD, STOOD-X, CUBIC.

EU AI Act: para sistemas de **alto riesgo** exige trazabilidad, explicabilidad, supervisión humana y documentación. La XAI ya no es opcional. **Desafíos:** explicaciones post-hoc pueden engañar (confirmation bias), sin métrica única, coste (SHAP exponencial), XAI multimodal y cadenas de agentes.

9.2 Trustworthy AI, AI Safety y detección OOD

- **Trustworthy AI (IA confiable):** lícita, ética y **robusta**. Es el conjunto de enfoques técnicos para diseñar **Responsible AI Systems** (auditables y con rendición de cuentas).

- **AI Safety:** prevenir accidentes, usos indebidos o consecuencias dañinas. **AI safety** ≠ **AI security**.
- **Robustez:** mantener el rendimiento ante perturbaciones y entradas adversarias durante todo el ciclo de vida (anomalías, ataques adversarios, puertas traseras, usos maliciosos).

Mundo cerrado vs mundo abierto (OOD)

- Los modelos asumen un **mundo cerrado** (todas las clases vistas en entrenamiento) → sobreocupan el espacio y no se adaptan a datos nuevos.
- **OOD (Out-of-Distribution):** detectar datos fuera de la distribución de entrenamiento (in-distribution vs out-of-distribution). Anomalías semánticas: *near* (CIFAR-100) vs *far* (EMNIST). **Open Set Recognition**.

9.3 Aprendizaje continuo (Continual Learning, CL)

- **Motivación:** el mundo es dinámico, reentrenar desde cero es inviable, hay límites de privacidad (RGPD), y los humanos aprenden de forma continua.
- **Definición:** aprender secuencialmente múltiples tareas ($T_1 \dots T_n$) sin reentrenar y manteniendo el rendimiento en las anteriores.
- **Olvido catastrófico (catastrophic forgetting):** al entrenar una tarea nueva, los gradientes sobrescriben los pesos de las anteriores (95% → 12%). Mayor diferencia entre tareas ⇒ mayor olvido (McCloskey & Cohen, 1989).

Tipos de CL

- **Task-Incremental:** se conoce el ID de tarea en test (el más sencillo).
- **Domain-Incremental:** misma tarea, distinta distribución; sin ID de tarea.
- **Class-Incremental:** nuevas clases en cada tarea; el más difícil (propenso al olvido).

Técnicas

- **EWC (Elastic Weight Consolidation):** regulariza ponderando cada peso por su importancia (Matriz de Información de Fisher): $L = L_B + \lambda \sum F_i(\theta_i - \theta^*_{A,i})^2$. Protege pesos importantes y libera los irrelevantes (mejor que ℓ_2 puro). Coste: calcular la FIM.
- **LoRA (Low-Rank Adaptation):** $W = W_0 + B \cdot A$; se congela W_0 y se entrenan matrices de bajo rango por tarea → sin olvido (W_0 no cambia), coste mínimo, compatible con LLMs. Variantes: O-LoRA, LoRA-Pro, Adapter Merging.
- **Métrica de olvido — Backward Transfer (BWT):** media de $A(N,j) - A(j,j)$. $BWT < 0$ olvido, $= 0$ sin olvido, > 0 transferencia positiva.

9.4 Machine Unlearning

- Problema inverso al CL: hacer que un modelo **olvide información específica** como si nunca la hubiera visto, sin reentrenar desde cero.
- **Motivación:** los modelos memorizan datos (ataques de extracción/pertenencia, Carlini 2020) y existe el **derecho al olvido** (RGPD Art. 17, AI Act 2024, LO 3/2018).
- **Tipos:** exacto vs aproximado; olvido de muestra vs olvido de clase.
- **Técnicas:**

- **Como CL:** tarea A = aprender, tarea B = olvidar (maximizar logit incorrecto, igualar logits, o distanciar en el espacio de embeddings).
- **SISA** (Sharded, Isolated, Sliced, Aggregated): divide datos en shards, entrena un modelo por shard con checkpoints; para olvidar, reentrena solo el shard/slice afectado.
- **SSD** (Selective Synaptic Dampening): amortigua los pesos importantes para D_{forget} pero no para D_{retain} usando la FIM (conexión con EWC). Aplicable a modelos ya entrenados.
- **Retos:** caída de rendimiento, unlearning secuencial, y **verificación** (¿en qué se diferencia un dato olvidado de uno nunca visto?).

9.5 Causalidad en la IA

- "**¿Pueden las vacas ir a la playa?**": un modelo entrenado con vacas en prados y camellos en desierto, al ver una vaca en la playa predice "camello" → **correlación espuria** (aprende el fondo, no el animal).
- **Correlación $\neq$ Causalidad:** helados y ahogamientos correlacionan por un **confounder** (el calor). Para afirmar causalidad se requiere **intervención**.
- **Causal Discovery:** inferir el grafo causal a partir de datos observacionales (dependencias e independencias condicionales). Requisitos: **suficiencia causal** (observar todos los confounders) y estructura **DAG** (grafo acíclico dirigido).
- **Causal Representation Learning:** descomponer el estado latente Z en Z_C (contenido invariante: la vaca) y Z_S (estilo/fondo espurio: la playa). Con supervisión débil e intervenciones en el estilo se aprenden representaciones invariantes.
- **V-REx (Variance Risk Extrapolation):** $R = \Sigma_e R_e + \beta \cdot \text{Var}(\{R_e\})$; minimiza el riesgo medio y la varianza entre entornos → representaciones invariantes que ignoran factores espurios.

Apéndice · Fórmulas clave y preguntas de repaso

A.1 Fórmulas imprescindibles

Concepto	Fórmula
RMSE	$\sqrt{((1/m)\sum(\hat{y}-y)^2)}$
MAE	$(1/m)\sum \hat{y}-y $
Precisión / Recall	$TP/(TP+FP)$ $TP/(TP+FN)$
F1	$2 \cdot P \cdot R / (P+R)$
Ecuación Normal	$\theta = (X^T X)^{-1} X^T y$
Gradiente MSE	$(2/m) X^T (X\theta - y)$
Sigmoide	$\sigma(z) = 1/(1+e^{-z})$
Softmax	$p_k = e^{s_k} / \sum e^{s_j}$
Naïve Bayes	$\operatorname{argmax}_c P(c) \cdot \prod P(a_i c)$
Entropía	$H = -\sum p_i \log_2 p_i$
Gini	$1 - \sum p_j^2$
Ganancia de info	$H(S) - \sum (S_v / S) H(S_v)$
Silueta	$(b-a)/\max(a,b)$
Atención	$\operatorname{softmax}(QK^T/\sqrt{d_k})V$
Salida convolución	$((N+2P-F)/\text{stride})+1$
Error generalización	$\text{Sesgo}^2 + \text{Varianza} + \text{Error irreducible}$

A.2 Preguntas de repaso (autoevaluación)

1. Define la terna (T, E, P) de Mitchell para un detector de fraude.
2. ¿Qué es el teorema "No Free Lunch" y qué implica en la práctica?
3. Diferencia entre sobreajuste y subajuste; cómo los detectan las curvas de aprendizaje.
4. Explica el compromiso sesgo-varianza y cómo lo afecta la complejidad del modelo.
5. ¿Cuándo usarías Ridge, Lasso o Elastic-Net? ¿Cuál hace selección de características?
6. Compara Batch GD, SGD y Mini-batch GD.
7. ¿Qué es el truco del kernel y por qué solo es posible en el problema dual de la SVM?
8. Calcula la ganancia de información de un atributo dado un pequeño conjunto.

9. Diferencia entre Gini y entropía; entre ID3, C4.5 y CART.
10. Explica bagging vs boosting y por qué el boosting no es paralelizable.
11. ¿Qué hace OOB en bagging? ¿Por qué ~37%?
12. Diferencia entre OvO y OvA y cuándo conviene cada uno.
13. Naïve Bayes: ¿por qué el suavizado de Laplace? Estima $P(x|c)$ para una variable numérica.
14. Enumera técnicas para tratar valores perdidos y para filtrar ruido (EF, CVCF, IPF).
15. Discretización supervisada vs no supervisada; ¿qué hace MDLP de Fayyad?
16. Filter vs wrapper en selección de características: ventajas e inconvenientes.
17. Explica la maldición de la dimensionalidad y cómo PCA usa la SVD.
18. k-Means: ¿qué es la inercia, el método del codo y la silueta? Limitaciones.
19. ¿Cómo funciona DBSCAN? ¿Qué es una instancia central?
20. Compara LOF, One-Class SVM e Isolation Forest para anomalías.
21. Describe el forward y backward pass de la retropropagación.
22. ¿Qué es el vanishing gradient y cómo lo mitigan He/Glorot, ReLU y Batch Norm?
23. Explica Momentum, AdaGrad, RMSProp y Adam.
24. ¿Cómo funciona Dropout y por qué regulariza? ¿Y MC Dropout?
25. Ventajas de las CNN (conectividad dispersa, weight sharing, equivarianza). Calcula el tamaño de salida y el n° de parámetros de una capa convolucional.
26. ¿Qué son Q, K, V? Escribe la fórmula de la atención escalada.
27. Self-attention vs cross-attention; ¿por qué hace falta positional encoding?
28. Compara LIME y SHAP. ¿Qué propiedades cumplen los valores de Shapley?
29. ¿Qué es la detección OOD y por qué importa el "mundo abierto"?
30. Explica el olvido catastrófico y cómo lo abordan EWC y LoRA.
31. ¿Qué es el machine unlearning? Describe SISA y SSD.
32. Correlación vs causalidad: explica el caso helados/ahogamientos y el rol del confounder.